ShopSmart

Server — TypeScript Edition

Design Patterns

Complete Implementation Guide

Version	1.0.0
Scope	Server only (Express + Prisma + TypeScript)
Patterns Covered	10 patterns (6 required + 4 supplemental)
Classification	Internal — Architecture Team

1. Overview & How to Use This Document

Purpose

This document is the definitive implementation guide for design patterns in the ShopSmart server codebase.

For each pattern you will find: (1) what it is and why it matters, (2) exactly which files in our server need it, (3) a before/after TypeScript code example, and (4) a precise implementation location table.

Follow this guide for every new feature and every refactoring session. Do not deviate from the patterns defined here without a team review.

1.1 Pattern Categories

Design patterns fall into three categories — **Creational** (how objects are created), **Structural** (how objects are composed), and **Behavioral** (how objects communicate). This guide covers all three as they apply to the ShopSmart server.

1.2 Complete Pattern Registry

#	Pattern	Category	Where in ShopSmart Server	Type	Priority
01	**Singleton**	*Creational*	PrismaClient, Logger, Config, Email Transporter	Creational	Critical
02	**Observer**	*Behavioral*	Order status events, Review moderation, Audit logging	Behavioral	Critical
03	**Factory**	*Creational*	Error creation, Service instantiation, Response builders	Creational	Critical
04	**Template Method**	*Behavioral*	Base route handler, Base service, Email templates	Behavioral	High
05	**Strategy**	*Behavioral*	Pricing/discounts, Auth token strategies, Email providers	Behavioral	Critical
06	**Adapter**	*Structural*	Prisma→Domain models, Nodemailer, External payment APIs	Structural	High
07	**Repository**	*Structural*	All Prisma queries → IUserRepo, IProductRepo, IOrderRepo	Structural	Critical
08	**Chain of Resp.**	*Behavioral*	Middleware pipeline: validate→auth→authorize→handler	Behavioral	High
09	**Decorator**	*Structural*	Logging decorator, Cache decorator, Rate-limit decorator	Structural	Medium
10	**Command**	*Behavioral*	Audit log commands, Async job queue, Order operations	Behavioral	Medium

1.3 Architectural Layers — Where Patterns Live

```
HTTP Request
     │
     ▼
┌──────────────────────────────────────────────────────────────┐
│  ROUTES LAYER  (auth.routes.ts, orders.routes.ts …)          │
│  Patterns: Chain of Responsibility (middleware pipeline)     │
└──────────────────────────────────────────────────────────────┘
                            │
     ▼
┌──────────────────────────────────────────────────────────────┐
│  CONTROLLER LAYER  (*.controller.ts)                         │
│  Patterns: Template Method, Factory (response/error)         │
└──────────────────────────────────────────────────────────────┘
                            │
     ▼
┌──────────────────────────────────────────────────────────────┐
│  SERVICE LAYER  (*.service.ts)                               │
│  Patterns: Strategy, Observer, Template Method, Command      │
└──────────────────────────────────────────────────────────────┘
                            │
     ▼
┌──────────────────────────────────────────────────────────────┐
│  REPOSITORY LAYER  (src/repositories/)                       │
│  Patterns: Repository, Adapter                               │
└──────────────────────────────────────────────────────────────┘
                            │
     ▼
┌──────────────────────────────────────────────────────────────┐
│  INFRASTRUCTURE  (Prisma, Nodemailer, JWT)                   │
│  Patterns: Singleton, Adapter                                │
└──────────────────────────────────────────────────────────────┘
```

01 Singleton Pattern

Creational

Definition

The Singleton pattern guarantees that a class is instantiated exactly once across the entire application lifetime. All consumers share the same instance. In Node.js this is critical for database connections, loggers, and configuration loaders — creating multiple instances wastes resources and causes inconsistent state.

Current State in ShopSmart Server

✕ CURRENT — Implicit, fragile singletons

```
// config/prisma.ts — works but
undocumented pattern,
// relies on globalThis hack, not an
explicit class
const globalForPrisma = globalThis as any;
export const prisma =
globalForPrisma.prisma
  ?? new PrismaClient({...});
if (process.env.NODE_ENV !== 'production')
  globalForPrisma.prisma = prisma;

// config/logger.ts — plain object, not a
class,
// can't be injected, mocked or extended
export const logger = {
  info: (msg) => console.log(...),
  error: (msg) => console.error(...),
};

// email.ts — transporter created at
module load,
// no lifecycle management, created twice
in tests
const transporter = env.SMTP_HOST
  ? nodemailer.createTransport({...}) :
null;
```

✓ AFTER — Explicit Singleton classes

```
// src/services/DatabaseService.ts
export class DatabaseService {
  private static instance: DatabaseService;
  private client: PrismaClient;

  private constructor() {
    this.client = new PrismaClient({
      log: env.isDev ? ['query','error'] :
[],
    });
  }
  static getInstance(): DatabaseService {
    if (!DatabaseService.instance) {
      DatabaseService.instance = new
DatabaseService();
    }
    return DatabaseService.instance;
  }
  get db(): PrismaClient { return
this.client; }
  async connect() { await
this.client.$connect(); }
  async disconnect() { await
this.client.$disconnect(); }
}
// Usage everywhere:
const db =
DatabaseService.getInstance().db;
```

All Singletons to Implement

Singleton Class	Responsibility	Current File	Priority
DatabaseService	Single PrismaClient — connects, disconnects, query logging	src/config/prisma.ts	**Critical**

LoggerService	Structured JSON logger — info/warn/error/debug levels	src/config/logger.ts	**Critical**
ConfigService	Validated env vars — throws on startup if missing required vars	src/config/env.ts	**High**
EmailTransporterService	Single nodemailer transporter — reused across all email sends	src/utils/email.ts	**High**

Complete Implementation

src/services/LoggerService.ts

```
export class LoggerService {
  private static instance: LoggerService;
  private readonly isProd = process.env.NODE_ENV === 'production';

  private constructor() {}

  static getInstance(): LoggerService {
    if (!LoggerService.instance)
      LoggerService.instance = new LoggerService();
    return LoggerService.instance;
  }

  info(msg: string, meta?: Record<string, unknown>) {
    if (!this.isProd) console.log(JSON.stringify({ level:'info', msg, ...meta }));
  }
  warn(msg: string, meta?: Record<string, unknown>) {
    if (!this.isProd) console.warn(JSON.stringify({ level:'warn', msg, ...meta
}));
  }
  error(msg: string, meta?: Record<string, unknown>) {
    console.error(JSON.stringify({ level:'error', msg, ...meta }));
  }
  debug(msg: string, meta?: Record<string, unknown>) {
    if (!this.isProd) console.debug(JSON.stringify({ level:'debug', msg, ...meta
}));
  }
}

// Export a pre-resolved instance for backward compat during migration
export const logger = LoggerService.getInstance();
```

src/services/ConfigService.ts

```
export class ConfigService {
  private static instance: ConfigService;
  private readonly config: ReturnType<typeof this.load>;

  private constructor() { this.config = this.load(); }

  static getInstance(): ConfigService {
    if (!ConfigService.instance)
      ConfigService.instance = new ConfigService();
    return ConfigService.instance;
```

```
  }

  private load() {
    const DATABASE_URL = process.env.DATABASE_URL;
    if (!DATABASE_URL) throw new Error('DATABASE_URL is required');
    return {
      DATABASE_URL,
      PORT: parseInt(process.env.PORT ?? '4000', 10),
      JWT_ACCESS_SECRET: process.env.JWT_ACCESS_SECRET ?? 'dev-secret',
      JWT_REFRESH_SECRET: process.env.JWT_REFRESH_SECRET ?? 'dev-refresh',
      JWT_REFRESH_EXPIRES_DAYS: parseInt(process.env.JWT_REFRESH_EXPIRES_DAYS ??
'30', 10),
      FRONTEND_URL: process.env.FRONTEND_URL ?? 'http://localhost:3000',
      // … all other env vars
    };
  }

  get<K extends keyof ReturnType<ConfigService['load']>>(key: K) {
    return this.config[key];
  }
}
```

Singleton Rules for ShopSmart

NEVER create more than the four singletons listed above. Most classes should NOT be singletons.

Service classes (AuthService, OrderService) are NOT singletons — they are created per-request or per-container registration.

In tests: use dependency injection to pass mock instances — never access the singleton directly in unit tests.

Thread safety is not a Node.js concern (single-threaded), but always guard the getInstance() method with a null check.

02 Observer Pattern

Behavioral

Define a one-to-many dependency so that when one object changes state, all dependents are notified automatically.

Definition

The Observer pattern establishes a publish-subscribe relationship. An event source (Subject/Publisher) maintains a list of interested parties (Observers/Subscribers). When something significant happens, the subject notifies all registered observers without knowing who they are. This decouples the event producer from event consumers — critical for our order lifecycle, audit logging, and notification system.

Why ShopSmart Server Needs It Now

✕ CURRENT — Tightly coupled side effects

```
// orders.routes.ts — mixed concerns
router.post('/', authenticate, async (req,
res) => {
  // 1. Create order
  const order = await
prisma.order.create({...});

  // 2. Send email — DIRECT CALL
  await
sendOrderConfirmationEmail(user.email,
order);

  // 3. Update inventory — DIRECT CALL
  await
prisma.product.updateMany({...decrement...});

  // 4. Write audit log — DIRECT CALL
  await prisma.auditLog.create({...});

  // Every new side effect = edit this
function!
  res.json({ success: true, data: order });
});
```

✓ AFTER — Decoupled via EventBus

```
// order.service.ts
async createOrder(dto: CreateOrderDTO) {
  const order = await
this.orderRepo.create(dto);

  // Emit ONE event — observers handle
the rest
  await
this.eventBus.emit('order.created', {
    orderId: order.id,
    userId: order.userId,
    total: order.total,
    items: order.items,
  });

  return order;
  // AuditLogObserver handles audit
logging
  // NotificationObserver sends
confirmation email
  // InventoryObserver decrements stock
  // Adding a new side effect = new
Observer class
}
```

EventBus Implementation

src/events/EventBus.ts

```
import { LoggerService } from '../services/LoggerService';

type EventHandler<T = unknown> = (payload: T) => Promise<void> | void;

export class EventBus {
  private static instance: EventBus;
```

```
  private readonly handlers = new Map<string, EventHandler[]>();
  private readonly logger = LoggerService.getInstance();

  private constructor() {}

  static getInstance(): EventBus {
    if (!EventBus.instance) EventBus.instance = new EventBus();
    return EventBus.instance;
  }

  subscribe<T>(event: string, handler: EventHandler<T>): void {
    const existing = this.handlers.get(event) ?? [];
    this.handlers.set(event, [...existing, handler as EventHandler]);
  }

  unsubscribe<T>(event: string, handler: EventHandler<T>): void {
    const existing = this.handlers.get(event) ?? [];
    this.handlers.set(event, existing.filter(h => h !== handler));
  }

  async emit<T>(event: string, payload: T): Promise<void> {
    const handlers = this.handlers.get(event) ?? [];
    await Promise.allSettled(
      handlers.map(async (h) => {
        try { await h(payload); }
        catch (err) {
          // Observers MUST NOT crash the main flow
          this.logger.error(`Observer failed for ${event}`, { err: String(err) });
        }
      })
    );
  }
}
```

src/events/events.types.ts — All domain events

```
export interface OrderCreatedEvent {
  orderId: string; userId: string; total: number; items: OrderItemDTO[];
}
export interface OrderStatusChangedEvent {
  orderId: string; userId: string; oldStatus: OrderStatus; newStatus: OrderStatus;
}
export interface UserRegisteredEvent {
  userId: string; email: string; fullName: string; verifyToken: string;
}
export interface ReviewSubmittedEvent {
  reviewId: string; userId: string; productId: string; rating: number;
}
export interface ReviewStatusChangedEvent {
  reviewId: string; productId: string; status: string;
}
export const EVENTS = {
  ORDER_CREATED:        'order.created',
  ORDER_STATUS_CHANGED: 'order.status_changed',
  USER_REGISTERED:      'user.registered',
  REVIEW_SUBMITTED:     'review.submitted',
  REVIEW_STATUS_CHANGED:'review.status_changed',
} as const;
```

src/events/observers/AuditLogObserver.ts

```
import { EventBus } from '../EventBus';
import { EVENTS } from '../events.types';
import { DatabaseService } from '../../services/DatabaseService';

export class AuditLogObserver {
  constructor(
    private readonly eventBus: EventBus,
    private readonly db = DatabaseService.getInstance().db,
  ) {}

  register(): void {
    this.eventBus.subscribe(EVENTS.ORDER_CREATED,
this.onOrderCreated.bind(this));
    this.eventBus.subscribe(EVENTS.ORDER_STATUS_CHANGED,
this.onOrderStatusChanged.bind(this));
    this.eventBus.subscribe(EVENTS.USER_REGISTERED,
this.onUserRegistered.bind(this));
    this.eventBus.subscribe(EVENTS.REVIEW_SUBMITTED,
this.onReviewSubmitted.bind(this));
  }

  private async onOrderCreated(payload: OrderCreatedEvent) {
    await this.db.auditLog.create({
      data: { action:'ORDER_CREATED', resource:'order',
              resourceId: payload.orderId, userId: payload.userId,
              metadata: { total: payload.total } },
    });
  }
  // … onOrderStatusChanged, onUserRegistered, onReviewSubmitted
}
```

src/events/observers/NotificationObserver.ts

```
import { EVENTS, OrderStatusChangedEvent, UserRegisteredEvent } from
'../events.types';

export class NotificationObserver {
  constructor(
    private readonly eventBus: EventBus,
    private readonly emailSvc: IEmailService,
  ) {}

  register(): void {
    this.eventBus.subscribe(EVENTS.USER_REGISTERED,
this.onUserRegistered.bind(this));
    this.eventBus.subscribe(EVENTS.ORDER_STATUS_CHANGED,
this.onOrderStatusChanged.bind(this));
  }

  private async onUserRegistered(p: UserRegisteredEvent) {
    await this.emailSvc.sendVerification(p.email, p.verifyToken);
  }

  private async onOrderStatusChanged(p: OrderStatusChangedEvent) {
    if (p.newStatus === 'SHIPPED')
      await this.emailSvc.sendOrderShipped(p.userId, p.orderId);
    if (p.newStatus === 'DELIVERED')
```

```
      await this.emailSvc.sendOrderDelivered(p.userId, p.orderId);
    }
  }
```

Observer Registration — Bootstrap

src/container.ts — Register observers on startup

```
const eventBus    = EventBus.getInstance();
const auditObs    = new AuditLogObserver(eventBus);
const notifObs    = new NotificationObserver(eventBus, emailService);
const inventObs   = new InventoryObserver(eventBus, productRepo);

auditObs.register();
notifObs.register();
inventObs.register();
// Adding a new side effect → new Observer class + one line here
```

Observer Class	Events Subscribed	Location	Priority
AuditLogObserver	order.created, order.status_changed, user.registered, review.submitted	src/events/observers/AuditLogObserver.ts	**Critical**
NotificationObserver	user.registered, order.status_changed (SHIPPED, DELIVERED)	src/events/observers/NotificationObserver.ts	**High**
InventoryObserver	order.created (decrement stockQty)	src/events/observers/InventoryObserver.ts	**High**
ReviewModerationObserver	review.submitted (auto-flag, notify admin)	src/events/observers/ReviewModerationObserver.ts	**Medium**

03 Factory Pattern

Creational

Define an interface for creating objects, letting subclasses or static methods decide which class to instantiate.

Definition

The Factory pattern centralises object creation. Instead of calling new AppError(400, 'Not found', 'NOT_FOUND') scattered across 30 files, a factory method enforces consistency and allows change in one place. We use three factories in ShopSmart: AppErrorFactory (error creation), ApiResponseFactory (response shape), and ServiceFactory (dependency wiring).

Current Problem

✕ CURRENT — new AppError() scattered everywhere

```
// admin.routes.ts
next(new AppError(404, 'User not found',
'NOT_FOUND'));

// orders.routes.ts
next(new AppError(400, 'items required',
'VALIDATION_ERROR'));

// cart.routes.ts
next(new AppError(404, 'Cart not found',
'NOT_FOUND'));

// super-admin.routes.ts
next(new AppError(409, 'User already
exists', 'CONFLICT'));

// Problems:
// - Status codes can drift (404 vs 400
for same condition)
// - Error codes must be memorised by
every developer
// - Changing 'NOT_FOUND' to
'RESOURCE_NOT_FOUND' = 20+ edits
// - No autocomplete on error types
```

✓ AFTER — AppErrorFactory

```
// src/factories/AppErrorFactory.ts
export class AppErrorFactory {
  static notFound(resource = 'Resource') {
    return new AppError(404, `${resource}
not found`, 'NOT_FOUND');
  }
  static conflict(msg = 'Already exists') {
    return new AppError(409, msg,
'CONFLICT');
  }
  static unauthorized(msg = 'Authentication
required') {
    return new AppError(401, msg,
'UNAUTHORIZED');
  }
  static forbidden(msg = 'Forbidden') {
    return new AppError(403, msg,
'FORBIDDEN');
  }
  static validation(msg: string) {
    return new AppError(400, msg,
'VALIDATION_ERROR');
  }
  static internal(msg = 'Internal server
error') {
    return new AppError(500, msg,
'INTERNAL_ERROR');
  }
}
// Usage:
next(AppErrorFactory.notFound('User'));
next(AppErrorFactory.conflict('Email
already registered'));
```

ApiResponseFactory

src/factories/ApiResponseFactory.ts

```
export interface ApiResponse<T = unknown> {
  success: boolean;
  data?: T;
  message?: string;
  meta?: { total?: number; page?: number; limit?: number; };
}

export class ApiResponseFactory {
  static success<T>(data: T, message?: string): ApiResponse<T> {
    return { success: true, data, ...(message && { message }) };
  }
  static paginated<T>(data: T[], total: number, page: number, limit: number):
ApiResponse<T[]> {
    return { success: true, data, meta: { total, page, limit } };
  }
  static created<T>(data: T): ApiResponse<T> {
    return { success: true, data };
  }
  static message(message: string): ApiResponse {
    return { success: true, message };
  }
}

// Usage in controllers:
res.json(ApiResponseFactory.success(user));
res.status(201).json(ApiResponseFactory.created(order));
res.json(ApiResponseFactory.message('Password reset.'));
```

ServiceFactory — Dependency Wiring

src/factories/ServiceFactory.ts

```
import { DatabaseService }   from '../services/DatabaseService';
import { PrismaUserRepository }   from '../repositories/PrismaUserRepository';
import { PrismaOrderRepository }  from '../repositories/PrismaOrderRepository';
import { NodemailerEmailService } from '../services/NodemailerEmailService';
import { BcryptHashService }      from '../services/BcryptHashService';
import { JwtTokenService }        from '../services/JwtTokenService';
import { AuthService }            from '../modules/auth/auth.service';
import { OrderService }           from '../modules/orders/order.service';

export class ServiceFactory {
  private static authService: AuthService;
  private static orderService: OrderService;

  private static get db() { return DatabaseService.getInstance().db; }

  static getAuthService(): AuthService {
    if (!this.authService) {
      this.authService = new AuthService(
        new PrismaUserRepository(this.db),
        new NodemailerEmailService(),
        new BcryptHashService(),
        new JwtTokenService(),
        EventBus.getInstance(),
      );
```

```
    }
    return this.authService;
  }

  static getOrderService(): OrderService {
    if (!this.orderService) {
      this.orderService = new OrderService(
        new PrismaOrderRepository(this.db),
        new PrismaProductRepository(this.db),
        EventBus.getInstance(),
      );
    }
    return this.orderService;
  }
}
```

Factory Class	Creates / Produces	Location	Priority
AppErrorFactory	Typed AppError instances with correct HTTP codes and error codes	src/factories/AppErrorFactory.ts	**Critical**
ApiResponseFactory	Standardised { success, data, message, meta } response shapes	src/factories/ApiResponseFactory.ts	**High**
ServiceFactory	Fully-wired service instances with injected repositories and deps	src/factories/ServiceFactory.ts	**Critical**

04
Behavioral

Template Method Pattern

Define the skeleton of an algorithm in a base class, deferring specific steps to subclasses.

Definition

The Template Method pattern defines the overall algorithm structure (the 'template') in a base class, while letting subclasses fill in the varying steps. In our server, every controller action follows the same skeleton: validate → call service → handle error → send response. Rather than repeating this structure in every route handler, we encode it once in a BaseController.

BaseController — The Main Template

src/base/BaseController.ts

```
import { Request, Response, NextFunction } from 'express';
import { AppError } from '../middleware/errorHandler';
import { ApiResponseFactory } from '../factories/ApiResponseFactory';
import { LoggerService } from '../services/LoggerService';

export abstract class BaseController {
  protected readonly logger = LoggerService.getInstance();

  // THE TEMPLATE — this algorithm never changes
  protected handleRequest<T>(
    handler: () => Promise<T>,
    res: Response,
    next: NextFunction,
    statusCode = 200,
  ): void {
    handler()
      .then((data) => {
        res.status(statusCode).json(ApiResponseFactory.success(data));
      })
      .catch(next);  // All errors go to errorHandler middleware
  }

  protected handleCreated<T>(
    handler: () => Promise<T>,
    res: Response,
    next: NextFunction,
  ): void {
    this.handleRequest(handler, res, next, 201);
  }

  protected handleMessage(
    handler: () => Promise<string>,
    res: Response,
    next: NextFunction,
  ): void {
    handler()
```

```
      .then((message) => res.json(ApiResponseFactory.message(message)))
      .catch(next);
  }
}
```

src/modules/auth/auth.controller.ts — Extends BaseController

```
export class AuthController extends BaseController {
  constructor(private readonly authService: AuthService) { super(); }

  register = (req: AuthRequest, res: Response, next: NextFunction): void => {
    this.handleCreated(
      () => this.authService.register(req.body),
      res, next,
    );
  };

  login = (req: AuthRequest, res: Response, next: NextFunction): void => {
    this.handleRequest(
      () => this.authService.login(req.body, req.ip),
      res, next,
    );
  };

  logout = (req: AuthRequest, res: Response, next: NextFunction): void => {
    this.handleMessage(
      async () => { await this.authService.logout(req.cookies.refreshToken);
return 'Logged out'; },
      res, next,
    );
  };
}
```

BaseEmailTemplate

src/base/BaseEmailTemplate.ts

```
export abstract class BaseEmailTemplate {
  // TEMPLATE METHOD — algorithm defined here
  render(data: Record<string, string>): string {
    const subject  = this.getSubject(data);      // step 1: subclass
    const greeting = this.getGreeting(data);      // step 2: subclass
    const body     = this.getBody(data);          // step 3: subclass
    const footer   = this.getFooter();            // step 4: default
    return this.wrap(subject, greeting, body, footer); // step 5: fixed
  }

  abstract getSubject(data: Record<string, string>): string;
  abstract getGreeting(data: Record<string, string>): string;
  abstract getBody(data: Record<string, string>): string;

  protected getFooter(): string {
    return '<p>— The ShopSmart Team</p>';
  }
```

```
  private wrap(subject: string, greeting: string, body: string, footer: string):
string {
    return `<html><body><h2>${greeting}</h2>${body}${footer}</body></html>`;
  }
}

// Subclass:
export class VerificationEmailTemplate extends BaseEmailTemplate {
  getSubject()  { return 'Verify your ShopSmart email'; }
  getGreeting(d){ return `Hello ${d.fullName}!`; }
  getBody(d)    { return `<p>Click to verify: <a
href='${d.link}'>${d.link}</a></p>`; }
}
```

Template Class	Fixed Algorithm / Variable Steps	Location	Priority
BaseController	handleRequest → validate → call service → respond → catch	src/base/BaseController.ts	**High**
BaseService	Common findById, exists, softDelete, paginate abstract methods	src/base/BaseService.ts	**High**
BaseEmailTemplate	render → getSubject + getGreeting + getBody + getFooter + wrap	src/base/BaseEmailTemplate.ts	**Medium**

05 Strategy Pattern

Behavioral

Define a family of algorithms, encapsulate each one, and make them interchangeable at runtime.

Definition

The Strategy pattern defines a family of interchangeable algorithms behind a common interface. The consuming class (Context) doesn't know which algorithm it's running — it just calls the interface method. In ShopSmart, pricing rules, discount calculations, email providers, and authentication token strategies are all ideal candidates.

Strategy 1 — Discount / Pricing

✕ CURRENT — Hardcoded pricing in routes

```
// orders.routes.ts
router.post('/', async (req, res) => {
  // Pricing logic INSIDE the route
handler
  let subtotal = 0;
  const orderItems = items.map((oi) => {
    const price = Number(product.price);
    subtotal += price * oi.quantity;
    return { productId: product.id,
             quantity: oi.quantity, price
};
  });
  // No coupon support, no discount logic,
  // adding them = editing this function
  const order = await
prisma.order.create({
    data: { subtotal, discount:0,
total:subtotal }
  });
});
```

✓ AFTER — Swappable strategy

```
//
src/strategies/discount/IDiscountStrategy.ts
interface IDiscountStrategy {
  calculate(subtotal: number, ctx:
DiscountContext): number;
}
// PercentDiscountStrategy.ts
class PercentDiscountStrategy implements
IDiscountStrategy {
  constructor(private readonly pct: number)
{}
  calculate(subtotal: number) {
    return subtotal * (this.pct / 100);
  }
}
// FixedDiscountStrategy.ts
class FixedDiscountStrategy implements
IDiscountStrategy {
  constructor(private readonly amount:
number) {}
  calculate(subtotal: number) {
    return Math.min(this.amount, subtotal);
  }
}
// NoDiscountStrategy.ts (null object)
class NoDiscountStrategy implements
IDiscountStrategy {
  calculate() { return 0; }
}
```

src/strategies/discount/DiscountStrategyFactory.ts

```
import { Coupon } from '@prisma/client';
import { IDiscountStrategy } from './IDiscountStrategy';
import { PercentDiscountStrategy } from './PercentDiscountStrategy';
import { FixedDiscountStrategy } from './FixedDiscountStrategy';
import { NoDiscountStrategy } from './NoDiscountStrategy';
```

```
export class DiscountStrategyFactory {
  static fromCoupon(coupon: Coupon | null): IDiscountStrategy {
    if (!coupon || !coupon.active) return new NoDiscountStrategy();
    switch (coupon.type) {
      case 'PERCENT': return new PercentDiscountStrategy(Number(coupon.value));
      case 'FIXED':   return new FixedDiscountStrategy(Number(coupon.value));
      default:        return new NoDiscountStrategy();
    }
  }
}

// OrderService usage:
const strategy = DiscountStrategyFactory.fromCoupon(coupon);
const discount = strategy.calculate(subtotal, { userId, items });
const total = subtotal - discount + shipping;
```

Strategy 2 — Shipping

src/strategies/shipping/IShippingStrategy.ts + implementations

```
interface IShippingStrategy {
  calculate(subtotal: number, addressCountry: string): number;
  getLabel(): string;
}

class FreeShippingStrategy implements IShippingStrategy {
  calculate() { return 0; }
  getLabel() { return 'Free Shipping'; }
}

class FlatRateShippingStrategy implements IShippingStrategy {
  constructor(private readonly rate: number) {}
  calculate() { return this.rate; }
  getLabel() { return `Flat ₹${this.rate}`; }
}

class FreeAboveThresholdStrategy implements IShippingStrategy {
  constructor(private readonly threshold: number) {}
  calculate(subtotal: number) { return subtotal >= this.threshold ? 0 : 49; }
  getLabel() { return `Free above ₹${this.threshold}`; }
}
// ShippingStrategyFactory reads active strategy from SystemConfig
// Super-admin can change shipping strategy via /super-admin/config/shipping-
providers
```

Strategy 3 — JWT Token (Auth)

src/strategies/auth/ITokenStrategy.ts + JwtStrategy

```
interface ITokenStrategy {
  signAccess(payload: AccessPayload): string;
  signRefresh(sub: string, jti: string): string;
  verifyAccess(token: string): AccessPayload;
```

```
  verifyRefresh(token: string): RefreshPayload;
}

// src/strategies/auth/JwtTokenStrategy.ts
export class JwtTokenStrategy implements ITokenStrategy {
  constructor(private readonly config: ConfigService) {}

  signAccess(payload: Omit<AccessPayload, 'type'>): string {
    return jwt.sign(
      { ...payload, type: 'access' },
      this.config.get('JWT_ACCESS_SECRET'),
      { expiresIn: 3600 }
    );
  }
  signRefresh(sub: string, jti: string): string {
    return jwt.sign(
      { sub, jti, type: 'refresh' },
      this.config.get('JWT_REFRESH_SECRET'),
      { expiresIn: this.config.get('JWT_REFRESH_EXPIRES_DAYS') * 86400 }
    );
  }
  verifyAccess(token: string)  { return jwt.verify(token,
this.config.get('JWT_ACCESS_SECRET')) as AccessPayload; }
  verifyRefresh(token: string) { return jwt.verify(token,
this.config.get('JWT_REFRESH_SECRET')) as RefreshPayload; }
}
```

Strategy Interface	Implementations	Location	Priority
IDiscountStrategy	PercentDiscount, FixedDiscount, NoDiscount, BuyXGetY (future)	src/strategies/discount/	**Critical**
IShippingStrategy	FreeShipping, FlatRate, FreeAboveThreshold, ByWeight (future)	src/strategies/shipping/	**High**
ITokenStrategy	JwtTokenStrategy (current), ApiKeyStrategy (future)	src/strategies/auth/	**High**
IEmailProviderStrategy	NodemailerStrategy, SendGridStrategy (future)	src/strategies/email/	**Medium**

06 Adapter Pattern

Structural

Convert the interface of a class into another interface clients expect. Let incompatible interfaces work together.

Definition

The Adapter pattern wraps an incompatible class to make it compatible with an expected interface. In our server, Prisma returns models with Decimal types, embedded relations, and private fields (passwordHash). Our controllers should never receive raw Prisma models — an Adapter converts them to clean DTOs before they leave the service layer.

The Core Problem

✕ CURRENT — Raw Prisma leaking to HTTP

```
// products.routes.ts — Decimal leaks to
API
const products = await
prisma.product.findMany({...});
// Manual conversion scattered everywhere:
const data = products.map((p) => ({
  ...p,
  price: Number(p.price),              //
manual
  originalPrice: p.originalPrice
    ? Number(p.originalPrice) : null,  //
manual
  subtotal: null,  // ???
  total: null,     // ???
}));

// orders.routes.ts — same manual
conversion
subtotal: Number(order.subtotal),
discount: Number(order.discount),
shipping: Number(order.shipping),
total: Number(order.total),
// This pattern repeated in 8+ route
files!
// passwordHash can accidentally slip into
response
```

✓ AFTER — Adapter handles all conversion

```
// src/adapters/ProductAdapter.ts
export class ProductAdapter {
  static toDTO(p: PrismaProduct & {
category: Category }): ProductDTO {
    return {
      id: p.id,
      name: p.name,
      slug: p.slug,
      description: p.description,
      price: Number(p.price),          //
Decimal → number
      originalPrice: p.originalPrice
        ? Number(p.originalPrice) : null,
      image: p.image,
      category: p.category,
      inStock: p.inStock,
      stockQty: p.stockQty,
      isNew: p.isNew,
      isDeal: p.isDeal,
      // NO: deletedAt, internal fields
    };
  }
  static toDTOList(ps: (PrismaProduct & {
category })[]): ProductDTO[] {
    return ps.map(this.toDTO.bind(this));
  }
}
```

src/adapters/UserAdapter.ts — Prevent passwordHash exposure

```
import { User as PrismaUser, Role } from '@prisma/client';

export interface UserDTO {
  id: string; email: string; fullName: string;
  role: string; avatarUrl: string | null;
  emailVerified: boolean; createdAt: string;
}
```

```
export class UserAdapter {
  static toDTO(user: PrismaUser & { role: Role }): UserDTO {
    return {
      id:            user.id,
      email:         user.email,
      fullName:      user.fullName,
      role:          user.role.name,
      avatarUrl:     user.avatarUrl,
      emailVerified: user.emailVerified,
      createdAt:     user.createdAt.toISOString(),
      // NEVER: passwordHash, resetTokenHash, emailVerifyToken
      // NEVER: failedLogins, lockedUntil (internal security fields)
    };
  }
}
// The current toUserResponse() function in auth.service.ts
// becomes UserAdapter.toDTO() — one canonical place
```

src/adapters/OrderAdapter.ts

```
export class OrderAdapter {
  static toDTO(order: PrismaOrderWithItems): OrderDTO {
    return {
      id:        order.id,
      status:    order.status,
      subtotal:  Number(order.subtotal),
      discount:  Number(order.discount),
      shipping:  Number(order.shipping),
      total:     Number(order.total),
      createdAt: order.createdAt.toISOString(),
      items:     order.items.map(i => ({
        id:       i.id,
        product:  i.product ? ProductAdapter.toDTO(i.product) : null,
        quantity: i.quantity,
        price:    Number(i.price),
      })),
    };
  }
}
```

src/adapters/PaymentGatewayAdapter.ts — External API Adapter

```
// Our domain interface
export interface IPaymentGateway {
  charge(amount: number, currency: string, orderId: string):
Promise<PaymentResult>;
  refund(paymentId: string, amount: number): Promise<RefundResult>;
}

// Adapts Stripe's SDK to our interface
export class StripeAdapter implements IPaymentGateway {
  constructor(private readonly stripe: Stripe) {}

  async charge(amount: number, currency: string, orderId: string):
Promise<PaymentResult> {
    const intent = await this.stripe.paymentIntents.create({
      amount: Math.round(amount * 100),  // Stripe uses cents
```

```
      currency,
      metadata: { orderId },
    });
    return { paymentId: intent.id, status: intent.status };
  }

  async refund(paymentId: string, amount: number): Promise<RefundResult> {
    const ref = await this.stripe.refunds.create({
      payment_intent: paymentId,
      amount: Math.round(amount * 100),
    });
    return { refundId: ref.id, status: ref.status };
  }
}
// Switching to Razorpay: new RazorpayAdapter implements IPaymentGateway
// OrderService never changes — it only knows IPaymentGateway
```

Adapter Class	Adapts From → To	Location	Priority
UserAdapter	Prisma User + Role → UserDTO (hides passwordHash, tokens)	src/adapters/UserAdapter.ts	**Critical**
ProductAdapter	Prisma Product (Decimal) → ProductDTO (number)	src/adapters/ProductAdapter.ts	**Critical**
OrderAdapter	Prisma Order + nested items → OrderDTO	src/adapters/OrderAdapter.ts	**Critical**
StripeAdapter	Stripe SDK API → IPaymentGateway interface	src/adapters/StripeAdapter.ts	**High**
NodemailerAdapter	Nodemailer transporter → IEmailService interface	src/adapters/NodemailerAdapter.ts	**High**

07
Structural

Repository Pattern

Abstract the data layer behind a collection-like interface — services never write raw database queries.

Definition

The Repository pattern provides a collection-like interface for accessing domain objects. Services treat the repository as an in-memory collection — they call userRepo.findByEmail(), productRepo.findAll(), orderRepo.create() without knowing anything about Prisma, SQL, or database schemas. This is the most impactful structural pattern for testability and maintainability.

The Problem — Prisma Everywhere

Currently, prisma is imported directly in 9 different files. This makes every test require a real database connection, and any Prisma API change cascades through dozens of functions.

src/repositories/user/IUserRepository.ts

```
import { User, Role } from '@prisma/client';
import { CreateUserDTO, UpdateUserDTO } from './user.types';

export interface IUserRepository {
  findById(id: string): Promise<(User & { role: Role }) | null>;
  findByEmail(email: string): Promise<(User & { role: Role }) | null>;
  findAll(opts?: { take?: number; skip?: number }): Promise<User[]>;
  create(data: CreateUserDTO): Promise<User & { role: Role }>;
  update(id: string, data: UpdateUserDTO): Promise<User & { role: Role }>;
  softDelete(id: string): Promise<void>;
  incrementFailedLogins(id: string): Promise<void>;
  resetLoginAttempts(id: string): Promise<void>;
  lockUntil(id: string, until: Date): Promise<void>;
  setVerifyToken(id: string, tokenHash: string, expires: Date): Promise<void>;
  markEmailVerified(id: string): Promise<void>;
  setResetToken(id: string, tokenHash: string, expires: Date): Promise<void>;
  clearResetToken(id: string): Promise<void>;
}
```

src/repositories/user/PrismaUserRepository.ts

```
import { PrismaClient } from '@prisma/client';
import { IUserRepository } from './IUserRepository';

export class PrismaUserRepository implements IUserRepository {
  constructor(private readonly prisma: PrismaClient) {}

  async findById(id: string) {
    return this.prisma.user.findUnique({
      where: { id, deletedAt: null },
```

```
      include: { role: true },
    });
  }

  async findByEmail(email: string) {
    return this.prisma.user.findFirst({
      where: { email: email.toLowerCase().trim(), deletedAt: null },
      include: { role: true },
    });
  }

  async create(data: CreateUserDTO) {
    return this.prisma.user.create({ data, include: { role: true } });
  }

  async softDelete(id: string): Promise<void> {
    await this.prisma.user.update({
      where: { id },
      data: { deletedAt: new Date(), active: false },
    });
  }

  async incrementFailedLogins(id: string): Promise<void> {
    await this.prisma.user.update({
      where: { id },
      data: { failedLogins: { increment: 1 } },
    });
  }
  // ... all other interface methods
}
```

Repository Interface	Prisma Implementation	Used By	Priority
IUserRepository	PrismaUserRepository	AuthService, UserService, SuperAdminService	**Critical**
IProductRepository	PrismaProductRepository	ProductService, OrderService, AdminService	**Critical**
IOrderRepository	PrismaOrderRepository	OrderService, AdminService, SuperAdminService	**Critical**
ICartRepository	PrismaCartRepository	CartService	**High**
IReviewRepository	PrismaReviewRepository	ReviewService, AdminService	**High**
ICouponRepository	PrismaCouponRepository	OrderService (discount validation)	**Medium**
IAuditLogRepository	PrismaAuditLogRepository	AuditLogObserver	**Medium**

08
Behavioral

Chain of Responsibility

Pass a request along a chain of handlers — each handler decides to process it or pass it to the next.

Definition

The Chain of Responsibility pattern forms a pipeline of handlers. Each handler can process the request, modify it, or pass it on. Our Express middleware pipeline is already a perfect Chain of Responsibility. The key is to formalise, document, and standardize the chain order so every route follows the same consistent pipeline.

ShopSmart Middleware Chain — Standardised Order

The enforced pipeline order for every protected route

```
// MANDATORY CHAIN ORDER — do not deviate

// Layer 1: Request Context (always first)
router.use(requestContextMiddleware);   // attaches requestId, startTime

// Layer 2: Security (always before auth)
router.use(helmet());                   // security headers
router.use(rateLimit({...}));           // rate limiting

// Layer 3: Parsing
router.use(express.json());             // body parsing
router.use(cookieParser());             // cookie parsing

// Layer 4: Authentication (who are you?)
router.use(authenticate);               // validates JWT, attaches req.user

// Layer 5: Authorization (can you do this?)
router.use(authorize(...ADMIN_ROLES)); // role check
// OR:
router.use(allowSelfOrAdmin('id'));     // self-or-admin check

// Layer 6: Validation (is the request well-formed?)
router.use(validate(schema));           // Zod/express-validator

// Layer 7: Business Handler
router.post('/', controller.create);    // the actual work

// Layer 8: Error Handler (always last, registered on app)
app.use(errorHandler);                  // converts AppError → JSON response
```

src/middleware/requestContext.ts — New middleware to add

```
import { v4 as uuidv4 } from 'uuid';
import { Request, Response, NextFunction } from 'express';
```

```
declare global {
  namespace Express {
    interface Request { requestId: string; startTime: number; }
  }
}

export function requestContextMiddleware(
  req: Request, _res: Response, next: NextFunction
): void {
  req.requestId = uuidv4();
  req.startTime = Date.now();
  next();
}

// Use in app.ts — must be the FIRST middleware registered
app.use(requestContextMiddleware);
```

Chain of Responsibility Rules

ALWAYS register middleware in the exact order shown above. Wrong order = security vulnerabilities.

NEVER put business logic inside middleware — middleware handles cross-cutting concerns only.

The errorHandler middleware MUST be the last app.use() call in app.ts. It must have all four parameters (err, req, res, next).

Each middleware must call next() to pass control, or next(error) to skip to error handler. Never both.

09
Structural

Decorator Pattern

Attach additional responsibilities to an object dynamically, as a flexible alternative to subclassing.

Definition

The Decorator pattern wraps an object to add new behavior without modifying the original class. In TypeScript, we implement this as higher-order functions (decorator factories) that wrap service methods to add logging, caching, or performance tracking transparently.

Logging Decorator

src/decorators/logging.ts

```
import { LoggerService } from '../services/LoggerService';

const logger = LoggerService.getInstance();

// Method decorator factory
export function WithLogging(label?: string) {
  return function (target: unknown, propertyKey: string, descriptor:
PropertyDescriptor) {
    const original = descriptor.value;
    descriptor.value = async function (...args: unknown[]) {
      const name = label ?? `${target?.constructor?.name}.${propertyKey}`;
      const start = Date.now();
      logger.debug(`[START] ${name}`, { args: args.length });
      try {
        const result = await original.apply(this, args);
        logger.debug(`[END] ${name}`, { duration: Date.now() - start });
        return result;
      } catch (err) {
        logger.error(`[ERROR] ${name}`, { err: String(err), duration: Date.now() -
start });
        throw err;
      }
    };
    return descriptor;
  };
}

// Usage on any service method:
class OrderService {
  @WithLogging('OrderService.createOrder')
  async createOrder(dto: CreateOrderDTO) { ... }
}
```

Decorator	Adds To	Location	Priority

@WithLogging()	Any service method — logs start/end/error/duration	src/decorators/logging.ts	**Medium**
@WithCache(ttl)	Read-heavy service methods — caches result in memory	src/decorators/cache.ts	**Medium**
@WithRateLimit()	High-risk operations — limits calls per userId per minute	src/decorators/rateLimit.ts	**Low**

10 Command Pattern

Behavioral

Encapsulate a request as an object — supporting undo, logging, and queueing of operations.

Definition

The Command pattern encapsulates an operation as a standalone object. This enables audit logging (every command is a record), queuing (process commands asynchronously), and undo/retry (re-execute a command). In ShopSmart, order mutations and admin actions are ideal commands — they produce audit trail entries automatically.

Implementation

src/commands/ICommand.ts + CreateOrderCommand

```
// Base command interface
export interface ICommand<TResult = void> {
  execute(): Promise<TResult>;
  getAuditEntry(): AuditEntry;
}

// src/commands/orders/CreateOrderCommand.ts
export class CreateOrderCommand implements ICommand<Order> {
  constructor(
    private readonly dto: CreateOrderDTO,
    private readonly orderRepo: IOrderRepository,
    private readonly userId: string,
  ) {}

  async execute(): Promise<Order> {
    return this.orderRepo.create(this.dto);
  }

  getAuditEntry(): AuditEntry {
    return {
      action:   'CREATE_ORDER',
      resource: 'order',
      userId:   this.userId,
      metadata: { items: this.dto.items },
    };
  }
}

// CommandBus executes command AND writes audit log
class CommandBus {
  async execute<T>(command: ICommand<T>): Promise<T> {
    const result = await command.execute();
    await auditRepo.create(command.getAuditEntry());
    return result;
  }
}
```

Command Class	Encapsulates	Location	Priority
CreateOrderCommand	Order creation with audit entry generation	src/commands/orders/CreateOrderCommand.ts	**Medium**
UpdateOrderStatusCommand	Status transition with validation and audit	src/commands/orders/UpdateOrderStatusCommand.ts	**Medium**
CreateAdminCommand	Admin user creation by super-admin	src/commands/admin/CreateAdminCommand.ts	**Low**
UpdateSystemConfigCommand	System config change with before/after audit	src/commands/config/UpdateSystemConfigCommand.ts	**Low**

11. Master Implementation Checklist

How to use this checklist

Work tasks in priority order: Critical → High → Medium → Low.
Each row shows the pattern number (#), exact task, the target file path, and priority level.
Create one PR per pattern group — do not mix patterns in a single pull request.
Before marking a task done: write or update unit tests that mock the injected dependencies.

#	Implementation Task	File / Location	Priority
01	Create DatabaseService singleton wrapping PrismaClient with connect/disconnect lifecycle	`src/services/DatabaseService.ts`	Critical
01	Convert logger object → LoggerService singleton class with structured log levels	`src/services/LoggerService.ts`	Critical
01	Create ConfigService singleton (wraps env.ts, validates on startup)	`src/services/ConfigService.ts`	High
01	Create EmailTransporterService singleton (single nodemailer transporter instance)	`src/services/EmailTransporterService.ts`	High
02	Create EventBus singleton — typed emit/subscribe for domain events	`src/events/EventBus.ts`	Critical
02	Define all domain events: OrderStatusChanged, ReviewSubmitted, UserRegistered, PaymentProcessed	`src/events/events.types.ts`	Critical
02	Create AuditLogObserver — subscribes to all mutating events, writes to AuditLog table	`src/events/observers/AuditLogObserver.ts`	Critical
02	Create NotificationObserver — sends emails on OrderStatusChanged, UserRegistered	`src/events/observers/NotificationObserver.ts`	High
02	Create InventoryObserver — decrements stockQty when OrderConfirmed event fires	`src/events/observers/InventoryObserver.ts`	High
02	Emit events in OrderService.create, OrderService.updateStatus, ReviewService.create	`src/modules/orders/order.service.ts + review.service.ts`	Critical
03	Create AppErrorFactory — static methods: notFound(), conflict(), unauthorized(), validation()	`src/factories/AppErrorFactory.ts`	Critical
03	Create ServiceFactory — instantiates and wires all service classes with dependencies	`src/factories/ServiceFactory.ts`	Critical

03	Create ApiResponseFactory — standardizes { success, data, message, meta } shape	`src/factories/ApiResponseFactory.ts`	High
03	Replace all 'new AppError(...)' calls with AppErrorFactory.xxx() in every route/service	`ALL route and service files`	Critical
04	Create BaseService abstract class with common findById, exists, softDelete methods	`src/base/BaseService.ts`	High
04	Create BaseController abstract class with handleRequest template method	`src/base/BaseController.ts`	High
04	Create BaseEmailTemplate abstract class — subclass for each email type	`src/base/BaseEmailTemplate.ts`	Medium
04	Extend BaseController in AuthController, ProductController, OrderController etc.	`src/modules/*/controllers/`	High
05	Create IDiscountStrategy interface + PercentDiscountStrategy + FixedDiscountStrategy	`src/strategies/discount/`	Critical
05	Create IShippingStrategy interface + FreeShippingStrategy + FlatRateShippingStrategy	`src/strategies/shipping/`	High
05	Create ITokenStrategy interface + JwtAccessStrategy + JwtRefreshStrategy	`src/strategies/auth/`	High
05	Create IEmailProviderStrategy + NodemailerStrategy + (stub) SendGridStrategy	`src/strategies/email/`	Medium
05	Inject DiscountStrategy into OrderService.create — read coupon type from Coupon.type	`src/modules/orders/order.service.ts`	Critical
06	Create UserAdapter — maps Prisma User model → domain UserDTO (no passwordHash exposed)	`src/adapters/UserAdapter.ts`	Critical
06	Create ProductAdapter — maps Prisma Product (Decimal) → domain ProductDTO (number)	`src/adapters/ProductAdapter.ts`	Critical
06	Create OrderAdapter — maps Prisma Order with nested items → OrderDTO	`src/adapters/OrderAdapter.ts`	Critical
06	Create PaymentGatewayAdapter interface + StripeAdapter stub	`src/adapters/PaymentGatewayAdapter.ts`	High
07	Create IUserRepository interface + PrismaUserRepository implementation	`src/repositories/user/`	Critical
07	Create IProductRepository interface + PrismaProductRepository implementation	`src/repositories/product/`	Critical

07	Create IOrderRepository interface + PrismaOrderRepository implementation	`src/repositories/order/`	Critical
07	Create ICartRepository interface + PrismaCartRepository implementation	`src/repositories/cart/`	High
07	Create IReviewRepository interface + PrismaReviewRepository implementation	`src/repositories/review/`	High
08	Document middleware chain order in src/middleware/README.md	`src/middleware/README.md`	Medium
08	Create RequestContextMiddleware — attaches requestId, timestamp to every req	`src/middleware/requestContext.ts`	High
09	Create withLogging() decorator factory for service methods	`src/decorators/logging.ts`	Medium
10	Create AuditCommand interface + CreateOrderCommand, UpdateOrderStatusCommand	`src/commands/`	Medium

12. Target Folder Structure

```
server/src/
├── adapters/                       ← Pattern 06: Adapter
│   ├── UserAdapter.ts
│   ├── ProductAdapter.ts
│   ├── OrderAdapter.ts
│   ├── NodemailerAdapter.ts
│   └── StripeAdapter.ts
├── base/                           ← Pattern 04: Template Method
│   ├── BaseController.ts
│   ├── BaseService.ts
│   └── BaseEmailTemplate.ts
├── commands/                       ← Pattern 10: Command
│   ├── ICommand.ts
│   ├── CommandBus.ts
│   └── orders/
│       ├── CreateOrderCommand.ts
│       └── UpdateOrderStatusCommand.ts
├── config/                         ← (existing — migrate to singletons)
│   ├── env.ts
│   ├── logger.ts
│   ├── prisma.ts
│   └── swagger.ts
├── decorators/                     ← Pattern 09: Decorator
│   ├── logging.ts
│   └── cache.ts
├── events/                         ← Pattern 02: Observer
│   ├── EventBus.ts
│   ├── events.types.ts
│   └── observers/
│       ├── AuditLogObserver.ts
│       ├── NotificationObserver.ts
│       └── InventoryObserver.ts
├── factories/                      ← Pattern 03: Factory
│   ├── AppErrorFactory.ts
│   ├── ApiResponseFactory.ts
│   └── ServiceFactory.ts
├── interfaces/                     ← SOLID: Interface Segregation
│   ├── IEmailService.ts
│   ├── IHashService.ts
│   ├── ITokenService.ts
│   └── INotificationService.ts
├── middleware/                     ← Pattern 08: Chain of Responsibility
│   ├── authenticate.ts
│   ├── authorize.ts
│   ├── errorHandler.ts
│   ├── validate.ts
│   ├── allowSelfOrAdmin.ts
│   └── requestContext.ts           ← NEW
├── modules/                        ← (existing — add controllers)
│   ├── auth/
│   │   ├── auth.routes.ts
│   │   ├── auth.controller.ts      ← NEW (extends BaseController)
│   │   ├── auth.service.ts         ← Refactor to class
│   │   └── auth.validation.ts
│   ├── orders/  (same structure)
│   ├── products/ (same structure)
```

```
│   └── ...
├── repositories/                    ← Pattern 07: Repository
│   ├── user/
│   │   ├── IUserRepository.ts
│   │   └── PrismaUserRepository.ts
│   ├── product/
│   ├── order/
│   ├── cart/
│   └── review/
├── services/                        ← Pattern 01: Singleton services
│   ├── DatabaseService.ts
│   ├── LoggerService.ts
│   ├── ConfigService.ts
│   ├── NodemailerEmailService.ts
│   ├── BcryptHashService.ts
│   └── JwtTokenService.ts
├── strategies/                      ← Pattern 05: Strategy
│   ├── discount/
│   │   ├── IDiscountStrategy.ts
│   │   ├── PercentDiscountStrategy.ts
│   │   ├── FixedDiscountStrategy.ts
│   │   ├── NoDiscountStrategy.ts
│   │   └── DiscountStrategyFactory.ts
│   ├── shipping/
│   │   ├── IShippingStrategy.ts
│   │   ├── FreeShippingStrategy.ts
│   │   └── FlatRateShippingStrategy.ts
│   └── auth/
│       └── JwtTokenStrategy.ts
├── app.ts
├── container.ts                     ← DI wiring, observer registration
└── server.ts
```

13. How the Patterns Work Together

Patterns do not work in isolation. The following walkthrough traces a single POST /orders request through the full pattern stack:

```
POST /api/orders

1. Chain of Responsibility (Pattern 08)
   requestContext → helmet → rateLimit → express.json → authenticate → validate →
handler

2. Controller — Template Method (Pattern 04)
   OrderController.create calls this.handleCreated()
   └── BaseController.handleRequest() defines the response algorithm

3. Service — Strategy + Factory (Patterns 05 + 03)
   OrderService.createOrder(dto)
   ├── DiscountStrategyFactory.fromCoupon(coupon)  → IDiscountStrategy.calculate()
   ├── ShippingStrategyFactory.fromConfig()        → IShippingStrategy.calculate()
   └── total = subtotal - discount + shipping

4. Repository (Pattern 07)
   this.orderRepo.create(orderData)   ← IOrderRepository
   └── PrismaOrderRepository wraps prisma.order.create()

5. Adapter (Pattern 06)
   OrderAdapter.toDTO(prismaOrder)    ← strips Decimal, formats dates

6. Observer — Event Emitted (Pattern 02)
   eventBus.emit('order.created', { orderId, userId, total })
   ├── AuditLogObserver  → writes to audit_logs table (Singleton DB)
   ├── NotificationObserver → emailSvc.sendOrderConfirmation()
   └── InventoryObserver → productRepo.decrementStock(items)

7. Factory — Response Built (Pattern 03)
   ApiResponseFactory.created(orderDTO)
   └── res.status(201).json({ success: true, data: orderDTO })

8. Singleton — Throughout (Pattern 01)
   DatabaseService.getInstance().db    — single PrismaClient
   LoggerService.getInstance()          — single structured logger
   EventBus.getInstance()               — single event bus
```

14. Non-Negotiable Implementation Rules

#	Rule	Rationale

1	**No import of prisma in route files**	Routes must call controllers. Controllers call services. Only repositories import PrismaClient.
2	**No new AppError() — use AppErrorFactory**	Consistency across all error codes and status codes. One place to change wording.
3	**No raw Prisma models in HTTP responses**	Use Adapters. passwordHash and internal tokens must never reach the API layer.
4	**All service classes use constructor injection**	DIP compliance. Makes unit testing with mocks possible without hitting a real database.
5	**Observer handlers must NOT throw to the EventBus**	Errors in observers must be caught internally. Side effects must never fail the main request.
6	**Exactly 4 singletons — no more**	DatabaseService, LoggerService, ConfigService, EmailTransporterService only. Services are NOT singletons.
7	**Strategies are stateless classes**	Strategy classes must not hold mutable state. Each calculate() call is pure and idempotent.
8	**Every new entity needs a Repository interface + Prisma implementation**	New Prisma model = new IXxxRepository + PrismaXxxRepository before writing service code.
9	**Middleware chain order is fixed**	validate → authenticate → authorize → handler. Deviating creates security holes.
10	**New side effects = new Observer, never edit existing service**	Adding push notifications = new PushObserver class. Never add push code to AuthService.

End of Design Patterns Documentation — ShopSmart Server v1.0.0

Read this document before writing any new server-side class or module.